

# Low-Bit Recurrent States in Hybrid Language Models

HONGREN CHEN

JIAYANG HE

hungjenchen@telswarm.com

1755147627@qq.com

Author preprint – September 25, 2026

Submitted to ICASSP 2027. Not yet peer reviewed.

## Abstract

Hybrid language models maintain fixed-size recurrent states, but existing quantizers typically use eight bits or more. Quantization errors persist according to channel decay rates. We derive distortion weights from the observability Gramian and combine them with normalized state ranges for mixed-precision bit allocation, without calibration data, rotation, or training. We also quantize decay rates logarithmically. With per-token state quantization, a four-bit mean payload reduces excess negative log-likelihood by factors of 3.3–27.9 relative to the best of seven baselines across three hybrid models; metadata costs vary. At six bits, negative log-likelihood differs from the FP32-state baseline by less than 0.005 nats. Ablations separate gains from variable bit widths, decay weighting, and range normalization. With less frequent write-backs, gains diminish and depend on the model and budget.

**Keywords:** linear attention, delta rule, state-space models, quantization, bit allocation

## 1 Introduction

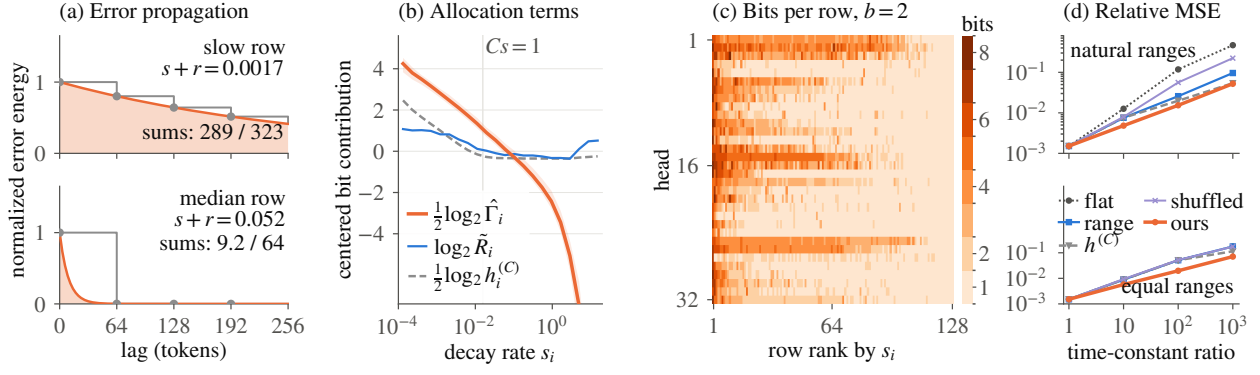
Gated linear recurrences retain information in fixed-size states [1, 2, 3]. Kimi Delta Attention (KDA) multiplies row  $i$  by  $e^{-s_i}$  per token, with time constant  $1/s_i$  for constant  $s_i$  [4]. Gated DeltaNet and Mamba-2 share decay within each head [3, 2]; Gated DeltaNet-2 separates erasing and writing [5]. Recurrent states commonly remain FP32 [6], even when projections use NVFP4 [7]; Nemotron 3 uses FP16 states with stochastic rounding [8].

Range-based quantization controls error at a single write-back, as in KV-cache compression [9]. Recurrent-state errors also affect later readouts: rapidly decaying rows dissipate errors, whereas slowly decaying rows accumulate them across write-backs.

Under a linear noise model, an observability Gramian determines accumulated readout distortion. For diagonal decay, its diagonal entries sum query-weighted squared decay factors (Fig. 1a). Combining these with normalized state ranges yields a classical high-rate bit-allocation rule [10, 11] without calibration data or training. We isolate decay-weighting gains under frequent write-back on three hybrid models, using excess negative log-likelihood (NLL).

**Related work.** DAMP [6] multiplies each key row’s low-precision error energy by a capped version of  $w^{(1)} = 1/(1 - a^2)$ , with effective decay  $a$ . Retaining 16 of 128 rows per head in FP16 and the rest in Hadamard INT8 requires 9.875 bits per element; INT4 and NVFP4 severely degrade accuracy. DASC [12] omits short-horizon rows from prefix checkpoints; deferred write-back reduces memory traffic [13, 14, 15]. We adapt recurrent-state quantizers [16, 17, 18, 8], KV-cache quantizers [19, 9, 20, 21, 22, 23], and four-bit formats [24, 7, 25] as baselines.

Persistence weighting [6] and rate-distortion allocation [26, 27] are established. We connect persistence to readout distortion, normalize ranges within output normalization groups, and evaluate



**Figure 1. Error propagation and bit allocation.** (a) Squared decay of an injected state error in two Kimi-Linear rows; gray steps sample it at write-backs. Paired sums refer to the decay curve and steps. (b) Centered contributions to the bit-allocation target. (c) Allocated bit widths in one layer. (d) Relative readout MSE in a synthetic recurrence with natural or equal row ranges. Settings:  $C = 64$ , two-bit mean payload.

lower-precision variable-width allocation and logarithmic decay-rate quantization. Without erasure, our weight is DAMP’s uncapped persistence times  $e^{-2s_i}$ ; their held-out NLL differs by at most 0.003 nats with chunked write-back.

## 2 Propagation of state quantization error

With  $\|\mathbf{k}_t\|_2 = 1$ ,  $\mathbf{D}_t = \text{diag}(e^{-s_t})$ , and  $\mathbf{s}_t > \mathbf{0}$ , each head’s state  $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$  follows

$$\begin{aligned} \mathbf{S}_t &= (\mathbf{I} - \mathbf{k}_t \mathbf{a}_t^\top) \mathbf{D}_t \mathbf{S}_{t-1} + \mathbf{k}_t (\mathbf{w}_t \odot \mathbf{v}_t)^\top, \\ \mathbf{o}_t &= \mathbf{S}_t^\top \mathbf{q}_t, \end{aligned} \quad (1)$$

where  $\mathbf{a}_t$  and  $\mathbf{w}_t$  control erasure and writing. KDA and Gated DeltaNet use  $\mathbf{a}_t = \beta_t \mathbf{k}_t$ ,  $\mathbf{w}_t = \beta_t \mathbf{1}$  [4, 3]; Mamba-2 has no erasure [2].

**Accumulated readout error.** Write-back occurs every  $C$  tokens: once per prefill chunk, per token in standard decoding, or less frequently with deferred write-back ( $C = 8$  in [15]). Outputs precede quantization, so an error introduced at  $\tau$  is first read at  $\tau + 1$  and propagates through  $\mathbf{F}_t = (\mathbf{I} - \mathbf{k}_t \mathbf{a}_t^\top) \mathbf{D}_t$  between write-backs.

**Proposition 1** (Time-averaged readout distortion). *Assume stationary layer inputs independent of quantization noise and finite accumulated error energy. Conditional on these inputs, noise is zero-mean, uncorrelated across rows and write-backs, with per-element row variance  $\sigma_i^2$ . Steady-state readout MSE per value column, averaged over positions within a write-back interval, is  $\sum_i \Gamma_i \sigma_i^2$ , where*

$$\Gamma_i = \frac{1}{C} \sum_{\ell \geq 1} \mathbb{E}[(\Phi_\ell^\top \mathbf{q}_{\tau+\ell})_i^2], \quad (2)$$

$\Phi_\ell = \mathbf{F}_{\tau+\ell} \cdots \mathbf{F}_{\tau+1}$ , and  $C\Gamma_i$  is a diagonal entry of the expected observability Gramian. Constant diagonal decay without erasure gives  $\Gamma_i = \rho_i / (C \expm1(2s_i))$ , with  $\rho_i = \mathbb{E}q_i^2$  and  $\expm1(x) = e^x - 1$ . For constant per-head decay and  $\mathbf{a} = \beta \mathbf{k}$ ,  $\beta \in [0, 2]$ ,  $\Gamma_i \leq \mathbb{E}\|\mathbf{q}\|_2^2 / (C \expm1(2s))$ .

*Proof sketch.* Cross terms vanish. Each write-back contributes at all lags  $\ell \geq 1$ , once per  $C$  tokens. Apply  $\sum_{\ell \geq 1} e^{-2s\ell} = 1/\expm1(2s)$  and  $\|\mathbf{I} - \beta \mathbf{k} \mathbf{k}^\top\|_2 \leq 1$ .  $\square$  The factor  $1/C$  scales distortion but

leaves allocation unchanged. Exogenous inputs are a local approximation: teacher forcing does not eliminate cross-layer noise propagation.

**Correlated rounding errors.** Deterministic rounding at  $C = 1$  can suppress updates below half a quantization step. Simulations with Kimi decays, rank-one writes, and 2K tokens show lag-one error correlations up to 0.23 for  $s_i \leq 10^{-3}$  at three to five bits. Readout error is 0.17–2.4 times that of independent noise with variance  $\Delta^2/12$  (step size  $\Delta$ ), with discrepancies within 20% from six bits. Proposition 1 is therefore a decoding heuristic: in these simulations, the model overestimates the gain over uniform four-bit allocation by 1.7–2.6 times.

**Approximate erasure rate.** For channel-wise decay, we approximate  $\Gamma_i \approx \rho_i/[C \expm1(2(s_i + r_i))]$ , with  $r_i = \frac{1}{2}\mathbb{E}[\beta(2 - \beta)k_i^2]$ . This is not a bound: errors orthogonal to a repeated key are never erased. With a fixed key,  $\mathbf{a} = \mathbf{k}/2$ ,  $s = 0.01$ , and  $C = 1$ , the exact and approximate  $\Gamma_i$  are 24.9 and 2.06. For simulated  $s$  log-uniform in  $[10^{-3}, 1]$ , allocation distortion differs from the exact-Gramian allocation by at most 3% with diverse keys and 17–23% with persistent keys. Omitting  $r_i$  gives comparable held-out NLL; we retain it.

**Range normalization.** To first order, RMSNorm projects out radial perturbations and divides by the group’s output RMS. Ignoring the projection and learned gains motivates normalizing ranges within each output normalization group: one head for KDA and Gated DeltaNet, or 12 heads for Nemotron-3-Nano’s Mamba-2 layers [28]. Assuming uniform  $\rho_i$  and output mean square proportional to the group’s mean squared row range gives the proxy  $\Gamma_i \tilde{R}_i^2$ . We evaluate these assumptions through ablations (Sec. 4); they are not exact RMSNorm properties.

### 3 Decay-aware quantization

**Decay-gate quantization.** A rate error  $\Delta s$  changes the signal retained at the time constant  $1/s$  by  $e^{-\Delta s/s}$ . The decay weight’s log-sensitivity also tends to  $-1$  as  $s \rightarrow 0$ , motivating levels uniform in  $\log s$  per channel. At two bits, gate-only quantization raises WikiText perplexity by at most 0.27 on Qwen3.5-0.8B/2B/9B, Qwen3.8-27B, and Kimi. It outperforms levels uniform in  $g = -s$  on all models; the latter give perplexity 86.9 on Kimi even at four bits, versus 9.33 in full precision. Subsequent experiments keep gates in full precision. With both gates and states at two bits, held-out NLL increases are approximately additive ( $C = 64$ : gates +0.037, states +0.006, both +0.047).

**State bit allocation.** We minimize the high-rate proxy  $\sum_i c_i 2^{-2b_i}$  using

$$c_i = \hat{\Gamma}_i \tilde{R}_i^2, \quad \hat{\Gamma}_i = \frac{1}{\expm1(2(\hat{s}_i + r_i))}. \quad (3)$$

Here  $\tilde{R}_i = R_i / \text{RMS}_{j \in u(i)} R_j$  normalizes ranges within output group  $u(i)$ ;  $R_i$  is the row absolute maximum, averaged over rows for a head unit. At layer-wise mean budget  $\bar{b}$ , the unconstrained real-valued solution is

$$b_i = \bar{b} + \frac{1}{2} \log_2 (c_i / \text{GM}(c)). \quad (4)$$

Under this proxy, the distortion reduction over uniform allocation is  $\text{AM}(c)/\text{GM}(c)$ , where AM and GM are the arithmetic and geometric means [10, 11]. This optimality does not extend to low-bit integer quantizers. We round, clip to  $[b_{\min}, b_{\max}]$ , and adjust widths to meet the layer budget, with  $b_{\max} = 8$  (16 at  $C = 1$ ). At each write-back,  $\hat{s}_i = -\frac{1}{2} \log \text{EMA}_{64}(e^{2g_{t,i}})$  and  $r_i = \frac{1}{2} \text{EMA}_{64}[\beta_t(2 - \beta_t)k_{t,i}^2]$ , using a chunk-aware exponential moving average with target span 64 tokens. In simulations with log-normal gates and widths in  $[1, 8]$ , this increases readout error by at most 2.5% over exact-Gramian allocation across tested  $C$ . A one-token estimate gives comparable decoding NLL.

**Allocation units.** For channel-wise decay (KDA) or normalization spanning heads (Mamba-2), each row has a scale and width, with  $b_{\min} = 1$ . When decay and normalization are per head (Gated

DeltaNet), each head has one width and scales per block of 32 key channels [19]. Then  $\tilde{R} = 1$  and only decay determines widths. We use  $b_{\min} = 2$  to avoid one-bit distortion of an entire head; a two-bit mean reduces to uniform block quantization.

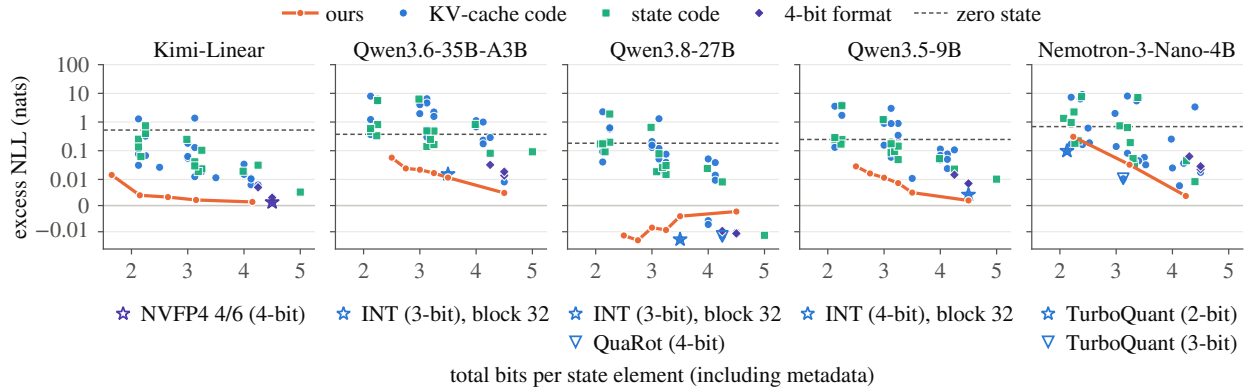
**Levels and storage.** For  $b \geq 2$ , we use levels  $\{-n, \dots, n\} \Delta$ , with  $n = 2^{b-1} - 1$  and  $\Delta = fR/n$ . We fit  $f$  over 16 values in  $[1/4, 1]$  by squared error per scale group, storing the result in an FP16 scale. One bit stores the sign times its mean absolute value. We count ternary levels as two bits, without ternary packing. An FP16 scale and three-bit width add  $19/d_v$  bits per row element ( $20/d_v$  with four-bit widths at  $C = 1$ ). Block scales add 0.5 bits per element; per-head widths are also counted. Prefix sums of widths give row offsets.

## 4 Bit-allocation ablations

We use expert-pruned [29] Kimi-Linear-REAP-35B-A3B-Instruct (20 KDA layers [4]), fake-quantized at each write-back in the released kernel. Unless specified, the mean payload is two bits. Excess NLL is measured against FP32 states on WikiText [30] in 2K-token windows. The rule was selected on the first 32K tokens at  $C = 64$ ; Table 1 uses tokens from 128K onward, and a second held-out set spans 32K–64K. Intervals are paired 95% bootstraps of our NLL minus the alternative’s; “tie”

**Table 1. Held-out bit-allocation ablations** on Kimi (excess NLL, nats). Only  $c_i$  varies. Payload: 4 bits at  $C = 1$ , 2 bits otherwise. Settings: 2K-token windows (8 at  $C = 1$ , else 32). Bold: minimum and entries whose paired 95% interval against it includes zero; –: not evaluated.

| <i>weight</i> $c_i$                                    | $C = 1$       | $C = 16$      | $C = 64$      |
|--------------------------------------------------------|---------------|---------------|---------------|
| $R_i^2$ , range alone                                  | +0.047        | +0.030        | +0.009        |
| $\hat{\Gamma}_i R_i^2$ , raw range                     | –             | +0.034        | +0.012        |
| $\tilde{R}_i^2$ , relative range                       | +0.072        | +0.043        | +0.011        |
| $w_i^{(1)} \tilde{R}_i^2$ , persistence                | –             | <b>+0.023</b> | +0.009        |
| $h_i^{(C)} \tilde{R}_i^2$ , write-back factor          | –             | <b>+0.019</b> | <b>+0.002</b> |
| $\hat{\Gamma}_i \tilde{R}_i^2$ , erase $r_i = 0$       | –             | <b>+0.021</b> | +0.006        |
| $\hat{\Gamma}_{\pi(i)} \tilde{R}_i^2$ , shuffled decay | –             | +0.082        | +0.045        |
| $\hat{\Gamma}_i \tilde{R}_i^2$ (ours)                  | <b>+0.034</b> | <b>+0.022</b> | +0.006        |
| uniform bit widths                                     | –             | +0.167        | +0.081        |



**Figure 2. Chunked state quantization** (WikiText,  $C = 64$ ). Excess NLL is relative to FP32 on a symmetric logarithmic scale. Stars and triangles identify configurations in the legends below each panel (nominal payloads).

means the interval includes zero.

**Effect of decay weighting.** Relative to allocation by raw range alone, our method reduces excess NLL by 0.013 at  $C = 1$  ( $[-0.021, -0.006]$ ), 0.008 at  $C = 16$  ( $[-0.013, -0.002]$ ), and 0.003 at  $C = 64$  ( $[-0.007, +0.000]$ , a tie). Relative to normalized range alone, the reductions are 0.038, 0.021, and 0.005; the last comparison ties on the second held-out set. This agrees qualitatively with  $1/C$  in (2). The observed gain ratio between  $C = 16$  and 64 is 4.0 versus normalized range but 1.9 versus uniform widths; its uncertainty is large (ratios 2–35), and  $C = 1$  uses four bits. Shuffling decay weights across rows worsens NLL in all 32 windows at both  $C = 16$  and 64, confirming the importance of matching weights to rows.

**Effect of range normalization.** Decay-only allocation is worse than ours on Kimi ( $-0.059$ ,  $[-0.072, -0.046]$  on the second set at  $C = 64$ ). Normalization preserves the ranking of rows within a head but changes allocation across heads. Slowly decaying heads tend to have larger ranges: the median layer-wise Spearman correlation between a head’s mean  $s_i$  and RMS range is  $-0.65$  (range  $-0.74$  to  $-0.18$ ). Normalized range alone removes this indirect decay signal and is 0.013 worse than raw range at  $C = 16$ . Combining decay with raw range can overweight these heads: it is worse than ours at both  $C = 16$  and 64 ( $-0.012$ ,  $[-0.016, -0.008]$  at  $C = 16$ ). On Nemotron, where normalization spans 12 heads, two-bit excess NLL is 1.92 with decay alone, 8.33 with raw range, 3.20 with normalized range, and 0.306 with the combined weight.

**Per-head allocation.** On Qwen3.6-35B-A3B [31], a one-bit floor and two-bit mean give excess NLL 1.42 with raw-range allocation, exceeding the zero-state baseline, and 0.219 with decay allocation, which is lower in all 16 windows. Both exceed the uniform block quantizer’s 0.057, motivating the two-bit floor. During decoding, decay weighting outperforms range-only allocation (Table 2). On Qwen3.5-9B at  $C = 64$ , decay allocation has lower NLL at all four budgets between 2.25 and 3 bits; the paired interval excludes zero at three bits.

**Choice of persistence factor.** With normalized ranges, both DAMP’s uncapped  $w_i^{(1)}$  and the write-back-time factor  $h_i^{(C)} = 1/(1 - e^{-2Cs_i})$  give NLL within 0.004 of ours. Both tie ours at  $C = 16$  and improve on normalized range alone by 0.020–0.024. At  $C = 64$ ,  $w_i^{(1)}$  is worse ( $-0.003$ ,  $[-0.005, -0.001]$ ), whereas  $h_i^{(C)}$  is better ( $+0.004$ ,  $[+0.0002, +0.008]$ ). Omitting the erasure rate ties at both intervals.

## 5 Experimental comparisons

**Protocol.** Each write-back is quantized and dequantized before reuse; no unquantized residual state is retained. Token-wise evaluation ( $C = 1$ ) uses Kimi, Qwen3.8-27B [32], and Qwen3.6-35B-A3B over eight 2K-token WikiText windows, overlapping the  $C = 64$  rule-selection data. Reference runs use  $C = 64$  and 32K tokens, adding Qwen3.5-9B [33] and Nemotron-3-Nano-4B (1K windows). Bits include scales, zero points, outlier indices, and width maps [9, 6]. TurboQuant is the best tested variant. DAMP-style uses DAMP’s score to assign INT8 to top-ranked rows within budget; the rest use Hadamard INT with  $\lfloor \bar{b} \rfloor - 1$  bits. Eight decoding windows limit bootstrap reliability. Quantizers are simulated, without kernel timings.

**Per-token state quantization** (Table 2). At a four-bit mean, ours outperforms all seven baselines in every window, reducing excess NLL by factors of 3.3, 4.8, and 27.9 relative to the best baseline on Kimi, Qwen3.8, and Qwen3.6, respectively. On Qwen3.8, TurboQuant uses 0.375 fewer bits; at equal storage, NVFP4 has 16 times our excess NLL. Range-only allocation with identical units and limits is 0.007 worse on Kimi (0.013 held out) and ties on Qwen3.8. On Qwen3.6, all four-bit baselines and range-only allocation ( $+1.15$ ) exceed the zero-state NLL increase ( $+0.373$ , resets every 64 tokens), versus our  $+0.086$ . DAMP-style’s low tier is INT3, rather than DAMP’s reported INT4 or INT8. At five and six bits, ours outperforms both uniform-width quantizers (one tie); six-bit excess NLL

**Table 2. Per-token state quantization** ( $C = 1$ ; excess NLL, nats). Ours outperforms all seven baselines in every window at four bits (unadjusted sign test  $p = 0.004$  per comparison). Settings: 8 WikiText windows of 2K tokens,  $b_{\max} = 16$ . INT uses fitted scales; range-only uses the same units as ours. Total bits: Kimi ( $\dagger 4.5$  on Qwen). Bold: minimum and statistical ties.

| (a) $\bar{b} = 4$          | bits            | Kimi          | Qwen3.8       | Qwen3.6       |
|----------------------------|-----------------|---------------|---------------|---------------|
| INT, row                   | 4.125           | +0.781        | +1.47         | +4.14         |
| INT, block 32              | 4.5             | +0.252        | +0.293        | +2.41         |
| TurboQuant [21]            | 4.125           | +0.131        | +0.073        | +2.83         |
| Hadamard INT, block 32 [6] | 5.0             | +0.135        | +0.142        | +4.44         |
| NVFP4 [7]                  | 4.5             | +0.236        | +0.242        | +2.62         |
| DAMP-style (INT3/8) [6]    | 5.016           | +0.222        | +1.79         | +8.68         |
| Stochastic rounding [8]    | 4.25            | +0.663        | +1.73         | +5.41         |
| range only                 | 4.156 $\dagger$ | +0.047        | <b>+0.024</b> | +1.15         |
| <b>ours</b>                | 4.156 $\dagger$ | <b>+0.040</b> | <b>+0.015</b> | <b>+0.086</b> |

| (b) $\bar{b}$ | Kimi          |               | Qwen3.8       |               | Qwen3.6       |               |
|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
|               | 5             | 6             | 5             | 6             | 5             | 6             |
| INT, row      | +0.174        | +0.027        | +0.571        | +0.338        | +2.03         | +1.22         |
| INT, block 32 | +0.075        | +0.034        | +0.010        | <b>−0.010</b> | +0.136        | +0.028        |
| <b>ours</b>   | <b>+0.014</b> | <b>+0.003</b> | <b>−0.001</b> | <b>−0.004</b> | <b>+0.021</b> | <b>+0.001</b> |

**Table 3. Chunked state quantization** ( $C = 64$ ; excess NLL, nats). Total bits are shown at  $\bar{b} = 2$  ( $\dagger$ Qwen3.6: DASC 2.234, ours 2.5). Bold: minimum and statistical ties. INT scales are fitted; Quamba2 uses static scales. Requirements: calibration (C), FP16 values (F), rotation (R). The zero-state baseline resets the state every 64 tokens.

| code                  | req. | bits               | Kimi-Linear   |               | Qwen3.6       |               |
|-----------------------|------|--------------------|---------------|---------------|---------------|---------------|
|                       |      |                    | $\bar{b}=2$   | 3             | $\bar{b}=2$   | 3             |
| zero state            | –    | 0                  | +0.525        |               | +0.373        |               |
| <i>KV-cache codes</i> |      |                    |               |               |               |               |
| INT, row              | –    | 2.125              | +0.076        | +0.034        | +0.365        | +0.299        |
| INT, block 32         | –    | 2.5                | +0.026        | +0.011        | <b>+0.057</b> | <b>+0.015</b> |
| KIVI, block 32 [19]   | –    | 3.0                | +0.182        | +0.034        | +4.85         | +1.12         |
| KVQuant, 1% [9]       | CF   | 3.0                | +0.055        | +0.017        | +1.96         | +0.761        |
| TurboQuant [21]       | R    | 2.125              | +0.031        | +0.012        | +1.22         | +0.516        |
| QuaRot [20]           | R    | 2.25               | +0.068        | +0.019        | +6.27         | +2.23         |
| OScaR [23]            | R    | 3.125              | +1.37         | +0.010        | +4.60         | +0.997        |
| <i>state codes</i>    |      |                    |               |               |               |               |
| Quamba2 [16]          | C    | 2.125              | +0.252        | +0.041        | +0.466        | +0.140        |
| DAMP-style [6]        | CR   | 2.984              | +0.249        | +0.019        | +6.32         | +0.825        |
| DASC-style [12]       | –    | 2.167 <sup>†</sup> | +0.062        | +0.019        | +0.332        | +0.250        |
| <b>ours</b>           | –    | 2.148 <sup>†</sup> | <b>+0.003</b> | <b>+0.002</b> | <b>+0.057</b> | <b>+0.012</b> |

is at most 0.0034. DAMP’s published setting uses 9.875 bits. At a four-bit mean, Kimi’s  $C = 64$  statistics assign 4% of rows more than eight bits, up to 14.

**Chunked state quantization** (Fig. 2, Table 3). For channel-wise decay, ours achieves the lowest measured NLL at 1.6–4.1 total bits per element and ties NVFP4 Four Over Six at a four-bit payload using 0.35 fewer total bits. For per-head decay at three and four bits, it leads on Qwen3.5-9B and ties the block quantizer on Qwen3.6. At four bits, it outperforms all tested four-bit formats on Qwen3.6, Qwen3.5-9B, and Nemotron.

**Long-context retrieval.** With a two-bit mean and  $b_{\max} = 8$ , write-backs occur every 64 prompt tokens and every answer token. On RULER [34] at 16K context (Kimi; 20 prompts per task), all quantizers and FP32 solve every multi-key query. For multi-value queries (four values per needle), recovery is 0.90 for ours, 0.80 for row-wise INT and TurboQuant, 0.76 for the fitted block, and 1.00 for FP32. Paired gains are 0.10 over INT and TurboQuant ( $[0.05, 0.15]$ ; no worse on any prompt), 0.14 over the 2.5-bit block ( $[0.08, 0.20]$ ), and  $-0.10$  against FP32 ( $[-0.15, -0.05]$ ).

**Limitations.** At  $C = 64$ , ours is best in four of 15 model–budget pairs, tied in seven (three identical to the block quantizer), and worse in four. On Qwen3.8, some quantizers have lower NLL than FP32: the three-bit block outperforms ours ( $+0.016$ ,  $[+0.003, +0.030]$ ), and at four bits five quantizers are better and one ties. Decay also loses to range-only allocation at two of four budgets. On Nemotron, nine quantizers are better at two bits (TurboQuant  $+0.098$ , ours  $+0.306$ ); Hadamard TurboQuant is better at three ( $+0.011$ , ours  $+0.033$ ). On held-out Kimi data at  $C = 64$  and three bits, normalized range outperforms ours ( $+0.009$ ,  $[+0.003, +0.015]$ ) and FP32, which the output-MSE model (2) cannot explain. At two bits,  $h_i^{(C)}$  is marginally better than ours (Table 1).

## 6 Conclusion

Decay weighting with normalized ranges improves low-bit recurrent-state quantization, especially with frequent write-backs. Correlated rounding and implementation costs require further study.

## 7 Acknowledgments

OpenAI Codex and Anthropic Claude Code assisted with language editing throughout the manuscript, research-code development, and idea discussions. The human authors originated the core claims, understand and have reviewed all content, and take full responsibility for it.

## 8 Compliance with Ethical Standards

This study comprises numerical simulations and evaluations of released models on public text benchmarks. It involves no human participants or animal experiments; ethical approval was not required.

## 9 Funding and Competing Interests

This independent research received no external or institutional funding. The authors have no relevant financial or nonfinancial interests to disclose.

## References

- [1] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [2] T. Dao and A. Gu, “Transformers are ssms: Generalized models and efficient algorithms through structured state space duality,” *arXiv preprint arXiv:2405.21060*, 2024.
- [3] S. Yang, J. Kautz, and A. Hatamizadeh, “Gated delta networks: Improving mamba2 with delta rule,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 29 687–29 707.

- [4] K. Team, Y. Zhang, Z. Lin *et al.*, “Kimi linear: An expressive, efficient attention architecture,” *arXiv preprint arXiv:2510.26692*, 2025.
- [5] A. Hatamizadeh, Y. Choi, and J. Kautz, “Gated deltanet-2: Decoupling erase and write in linear attention,” *arXiv preprint arXiv:2605.22791*, 2026.
- [6] T. Zhang, J. Tan, P. Sun *et al.*, “Damp: Decay-aware mixed-precision recurrent-state quantization,” *arXiv preprint arXiv:2608.27513*, 2026.
- [7] S. Kozyrev and D. Maiboroda, “Why gated deltanet survives 4-bit quantization: Nvfp4 w4a4 for the recurrent half of a hybrid 27b llm,” *arXiv preprint arXiv:2609.04098*, 2026.
- [8] A. Chandiramani, A. Blakeman, A. Olaoye *et al.*, “Nemotron 3 super: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning,” *arXiv preprint arXiv:2604.12374*, 2026.
- [9] C. Hooper, S. Kim, H. Mohammadzadeh *et al.*, “Kvquant: Towards 10 million context length llm inference with kv cache quantization,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 1270–1303, 2024.
- [10] J. Huang and P. Schultheiss, “Block quantization of correlated gaussian random variables,” *IEEE Transactions on Communications Systems*, vol. 11, no. 3, pp. 289–296, 1963.
- [11] A. Gersho and R. M. Gray, *Vector quantization and signal compression*. Springer Science & Business Media, 2012.
- [12] Y. Yu, P. Sun, J. Tan *et al.*, “Dasc: Decay-aware state compression for hybrid linear-attention serving,” *arXiv preprint arXiv:2608.30386*, 2026.
- [13] J. Lin, J. Sun, and G. Sun, “Deltalog: Deferred materialization of recurrent states for linear attention decoding,” *arXiv preprint arXiv:2608.15533*, 2026.
- [14] L. Zou and L. Zhong, “Kvbuffer: Io-aware serving for linear attention,” *arXiv preprint arXiv:2605.19049*, 2026.
- [15] A. Blakeman, A. Thomas, A. Jhunjhunwala *et al.*, “Nemotron 3 ultra: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning,” *arXiv preprint arXiv:2606.15007*, 2026.
- [16] H.-Y. Chiang, C.-C. Chang, N. Frumkin *et al.*, “Quamba2: A robust and scalable post-training quantization framework for selective state space models,” *arXiv preprint arXiv:2503.22879*, 2025.
- [17] H. Kim, B. Ko, M. Kang *et al.*, “Ssd8: Accurate and efficient 8-bit quantization for state space duality,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 97 014–97 042.
- [18] C. Tianqi, Y. Chen, P. Wang *et al.*, “Q-mamba: Towards more efficient mamba models via post-training quantization,” in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 10 594–10 610.
- [19] Z. Liu, J. Yuan, H. Jin *et al.*, “Kivi: A tuning-free asymmetric 2bit quantization for kv cache,” *arXiv preprint arXiv:2402.02750*, 2024.
- [20] S. Ashkboos, A. Mohtashami, M. L. Croci *et al.*, “Quarot: Outlier-free 4-bit inference in rotated llms,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 100 213–100 240, 2024.
- [21] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, “Turboquant: Online vector quantization with near-optimal distortion rate,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 56 418–56 439.
- [22] L. K. Muller, P. Bich, C. Borette *et al.*, “Kvarn: Variance-normalized kv-cache quantization mitigates error accumulation in reasoning tasks,” *arXiv preprint arXiv:2606.03458*, 2026.
- [23] Z. Su, R. Yang, C. Zhang *et al.*, “Oscar: The occam’s razor for extreme kv cache quantization in llms and beyond,” *arXiv preprint arXiv:2605.19660*, 2026.
- [24] B. D. Rouhani, R. Zhao, A. More *et al.*, “Microscaling data formats for deep learning,” *arXiv preprint arXiv:2310.10537*, 2023.

- [25] J. Cook, J. Guo, G. Xiao *et al.*, “Four over six: More accurate nvfp4 quantization with adaptive block scaling,” *arXiv preprint arXiv:2512.02010*, 2025.
- [26] H. Laus, C. M. Verdun, H. Wang *et al.*, “Kv cache compression through the lens of transform coding,” *arXiv preprint arXiv:2608.14191*, 2026.
- [27] K. Staniszewski and A. Lancucki, “Kv cache transform coding for compact storage in llm inference,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 37 768–37 815.
- [28] A. Taghibakhshi, S. T. Sreenivas, S. Muralidharan *et al.*, “Nemotron elastic: Towards efficient many-in-one reasoning llms,” *arXiv preprint arXiv:2511.16664*, 2025.
- [29] M. Lasby, I. Lazarevich, N. Sinnadurai *et al.*, “Reap the experts: Why pruning prevails for one-shot moe compression,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 146 883–146 912.
- [30] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [31] Q. Team, “Qwen3. 6-35b-a3b: Agentic coding power, now open to all,” 2026.
- [32] Qwen Team, “Qwen3.8-Max: A New Bar for Coding and Cowork,” August 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.8>
- [33] Qwen Team, “Qwen3.5: Towards native multimodal agents,” February 2026.
- [34] C.-P. Hsieh, S. Sun, S. Krizan *et al.*, “Ruler: What’s the real context size of your long-context language models?” *arXiv preprint arXiv:2404.06654*, 2024.